

Analisis de Patrones de Diseno y Arquetipos

Proyecto: Tienda Retail Lumina - Arquitectura de Microservicios

1. Introduccion

Este documento detalla los patrones de diseno y arquetipos seleccionados para la migracion de la plataforma Lumina desde una arquitectura monolitica hacia una basada en microservicios. La decision responde a problemas de saturacion en horas peak, buscando escalabilidad, mantenibilidad y desacoplamiento.

2. Patrones de Diseno Utilizados

2.1 Backend For Frontend (BFF)

Componente: bff-gateway (Puerto 8080)

Descripcion: El BFF actua como unica puerta de entrada entre el frontend y los microservicios. Centraliza las llamadas, orquesta respuestas y expone una API unificada bajo /api.

Justificacion: - Evita que el frontend conozca la ubicacion de cada microservicio. - Permite agregar logica de orquestacion (ej: combinar datos de productos + inventario). - Simplifica CORS y autenticacion al tener un solo punto de entrada. - Facilita cambios en los microservicios sin afectar al frontend.

Implementacion: Controladores Proxy en Spring Boot que redirigen peticiones HTTP a los microservicios internos usando RestTemplate/WebClient.

2.2 Database per Service (Base de Datos por Servicio)

Componentes: 6 instancias MySQL independientes (puertos 3307-3312)

Descripcion: Cada microservicio posee su propia base de datos MySQL, garantizando total independencia de datos.

Microservicio	Base de Datos	Puerto
ms-usuarios	db_usuarios	3307
ms-productos	db_productos	3308
ms-bodega	db_bodega	3309
ms-carrito	db_carrito	3310
ms-delivery	db_delivery	3311
ms-pagos	db_pagos	3312

Justificacion: - Desacoplamiento total: un cambio en el esquema de un servicio no afecta a otros. - Escalabilidad independiente: cada BD puede escalar segun la carga de su servicio. - Resiliencia: si una BD falla, los demas servicios siguen operando. - Cumple con el principio de responsabilidad unica.

2.3 Repository Pattern (Patron Repositorio)

Componentes: Interfaces JPA Repository en cada microservicio.

Descripcion: Se utiliza Spring Data JPA con interfaces JpaRepository para abstraer el acceso a datos, separando la logica de negocio de la persistencia.

Ejemplos: - `InventarioRepository` - acceso a tabla de inventario - `AlertaStockRepository` - acceso a alertas de stock - `UsuarioRepository` - acceso a usuarios - `ProductoRepository` - acceso a productos

Justificacion: - Reduce codigo boilerplate de acceso a datos. - Facilita testing con mocks. - Permite cambiar la implementacion de persistencia sin afectar la logica de negocio.

2.4 Service Layer Pattern (Capa de Servicio)

Componentes: Clases `*Service` en cada microservicio.

Descripcion: La logica de negocio se encapsula en clases de servicio (`@Service`), separandola de los controladores REST y los repositorios.

Ejemplos: - `BodegaService` - gestion de inventario, alertas de stock critico, descuento de stock. - `UsuarioService` - autentificacion, registro, gestion de perfiles. - `DeliveryService` - seguimiento de entregas, historial de estados.

Justificacion: - Separacion clara de responsabilidades (SRP). - Los controladores solo manejan HTTP request/response. - La logica de negocio es reutilizable y testeable.

2.5 DTO Pattern (Data Transfer Object)

Componentes: Clases `*Dto` en cada microservicio.

Descripcion: Se utilizan DTOs para transferir datos entre capas y entre servicios, evitando exponer directamente las entidades JPA.

Ejemplos: - `InventarioDto`, `AlertaStockDto` (ms-bodega) - `ProductoDto`, `MarcaDto` (ms-productos) - `UsuarioDto` (ms-usuarios) - `DeliveryDto`, `HistorialDeliveryDto` (ms-delivery) - `PagoDto` (ms-pagos) - `CarritoDto`, `ItemCarritoDto` (ms-carrito)

Justificacion: - Desacopla la representacion interna (entidades) de la API publica. - Permite controlar que datos se exponen al exterior. - Evita problemas de serializacion circular con JPA.

2.6 Builder Pattern

Componentes: Entidades con anotacion `@Builder` de Lombok.

Descripcion: Las entidades JPA usan el patron Builder (via Lombok) para construir objetos de forma legible y segura.

Ejemplo:

```
AlertaStock alerta = AlertaStock.builder()
    .productoId(inventario.getProductoId())
    .nombreProducto(inventario.getNombreProducto())
    .stockActual(inventario.getStock())
    .stockMinimo(inventario.getStockMinimo())
    .build();
```

Justificacion: - Construccin de objetos clara y sin ambigüedades. - Evita constructores con muchos parametros. - Inmutabilidad controlada.

2.7 Proxy Pattern

Componente: bff-gateway - Controladores Proxy.

Descripcion: El BFF implementa controladores que actuan como proxies, redirigiendo las peticiones del frontend al microservicio correspondiente.

Ejemplos: - `BodegaProxyController` → redirige a `ms-bodega:8083` - `ProductosProxyController` → redirige a `ms-productos:8082` - `UsuariosProxyController` → redirige a `ms-usuarios:8081`

Justificacion: - El frontend solo conoce una URL (`localhost:8080/api`). - Facilita agregar autentificacion,

rate limiting o caching a nivel de gateway. - Permite reemplazar o escalar microservicios de forma transparente.

2.8 Observer Pattern (Alertas de Stock)

Componente: BodegaService en ms-bodega.

Descripcion: El sistema monitorea los niveles de stock y genera/resuelve alertas automaticamente cuando el stock cruza el umbral minimo.

Comportamiento: - Stock <= stockMinimo → se genera alerta (o actualiza la existente). - Stock > stockMinimo → se resuelven todas las alertas activas del producto. - Se previenen alertas duplicadas verificando existencia previa.

Justificacion: - Notificacion automatica al administrador sin necesidad de consulta manual. - Ciclo de vida completo de alertas (creacion → actualizacion → resolucion).

3. Arquetipos Maven

3.1 Arquetipo Base: Spring Boot Microservice

Todos los microservicios backend se generaron siguiendo un arquetipo comun basado en **Spring Boot 3.2.5 con Java 17**.

Estructura estandar del arquetipo:

```
ms-{nombre}/
├── pom.xml                # Dependencias Maven
├── Dockerfile             # Multi-stage build
├── src/main/java/com/lumina/{nombre}/
├── Ms{Nombre}Application.java # Clase principal @SpringBootApplication
├── controller/            # Controladores REST (@RestController)
├── service/               # Logica de negocio (@Service)
├── repository/           # Acceso a datos (JpaRepository)
├── entity/               # Entidades JPA (@Entity)
└── dto/                  # Data Transfer Objects
```

Dependencias comunes del arquetipo:

Dependencia	Proposito
spring-boot-starter-web	API REST
spring-boot-starter-data-jpa	Persistencia ORM
spring-boot-starter-validation	Validacion de datos
mysql-connector-j	Driver MySQL
lombok	Reduccion de boilerplate
spring-boot-starter-test	Testing

Configuracion Maven comun: - groupId: com.lumina - parent: spring-boot-starter-parent:3.2.5 - java.version: 17 - Plugin: spring-boot-maven-plugin con exclusion de Lombok

3.2 Arquetipo BFF Gateway

El BFF usa un arquetipo similar pero sin JPA/MySQL, reemplazando por WebFlux para comunicacion reactiva con los microservicios.

Dependencias especificas: - spring-boot-starter-webflux (cliente HTTP reactivo) - spring-boot-starter-actuator (health checks)

4. Patrones de Frontend

4.1 Component-Based Architecture

El frontend utiliza **React** con componentes funcionales y hooks, organizados en: - pages/ - Paginas principales (AdminPage, CarritoPage, etc.) - components/ - Componentes reutilizables (Navbar, etc.) - context/ - Estado global (AuthContext) - services/ - Capa de comunicacion con el backend (api.js)

4.2 Context API Pattern

Se utiliza React Context (AuthContext) para manejar el estado de autenticacion global, evitando prop drilling.

4.3 Service Layer (Frontend)

El archivo api.js centraliza todas las llamadas HTTP al BFF, encapsulando Axios y exponiendo servicios tipados (productosService, bodegaService, carritoService, etc.).

5. Patron de Containerizacion

Docker Multi-Stage Build

Todos los microservicios usan un Dockerfile con multi-stage build:

```
# Etapa 1: Compilacion
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN apk add --no-cache maven && mvn clean package -DskipTests
```

```
# Etapa 2: Ejecucion (imagen liviana)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE {puerto}
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Justificacion: - Imagen final liviana (solo JRE, no JDK). - Compilacion reproducible dentro del contenedor. - Docker Compose orquesta todos los servicios con health checks.

6. Resumen de Patrones

Patron	Donde se aplica	Beneficio principal
BFF (Backend For Frontend)	bff-gateway	API unificada para el frontend
Database per Service	MySQL x6	Desacoplamiento de datos
Repository	Interfaces JPA	Abstraccion de persistencia
Service Layer	Clases @Service	Separacion de logica de negocio
DTO	Clases *Dto	Control de datos expuestos
Builder	Entidades Lombok	Construccion limpia de objetos
Proxy	BFF Controllers	Redireccion transparente
Observer	Alertas de stock	Notificaciones automaticas
Component-Based	React Frontend	UI modular y reutilizable
Multi-Stage Build	Dockerfiles	Imagenes optimizadas